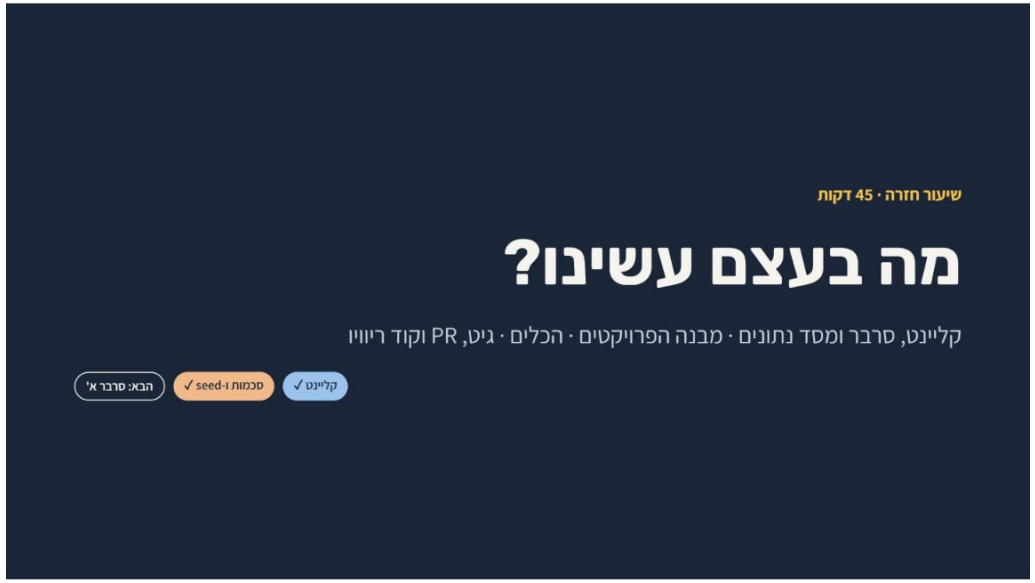


מטרת השיעור, להגיד בקול: "סיימנו שתי משימות גדולות. עכשיו עוצרות ומחברות את הנקודות. המטרה היא לא ללמוד משהו חדש. המטרה היא שכל אחת תוכל להסביר לחברה מה היא עשתה לבקש מהבנות לסגור מחשבים בשלב הזה. הן יפתחו אותם בחלק של גיט, להדגיש שזה בסדר לא להבין עדיין. יש חלקים שעשינו לפי הוראות, וזה נורמלי בהתחלה. היום נשלים את ה"למה". ולמה." ושוב אחרי השיעור כשממזגות.



החלק "הפלואו שלי" חשוב אם תעשי את כל ההעמקות בכל החלקים, יצא יותר משעה. הדקות בטבלה הן טווח: המספר הקטן הוא רק הבסיס והבדיקה, והמספר הגדול הוא עם כל ההעמקות. לכן המדחום בשקף הבא קובע איפה להעמיק ואיפה לדלג, כדי במיוחד: עדיף לקצר בכלים ובמבנה ולא בו, ומה שלא נכנס (כרטיס המסך, המעקב אחרי פעולה) הופך לשיעורי בית לפני סרבר א'. בדיקה שעברה טוב = ממשיכות. בסוף רוב החלקים יש עצירת בדיקה (שקף כהה). אחריו מגיעים שקפי העמקה (תגית בהירה). הכלל: בכל חלק מציגות את שקף הבסיס (תגית כהה). להגיע ל-45. אפשר לקצר בכלים ובמבנה. אם הזמן לחוץ, לא לדלג על החלק של גיט. בדיקה שנכשלה = חוזרות לשקף ההעמקה שמופיע בהערות של הבדיקה.

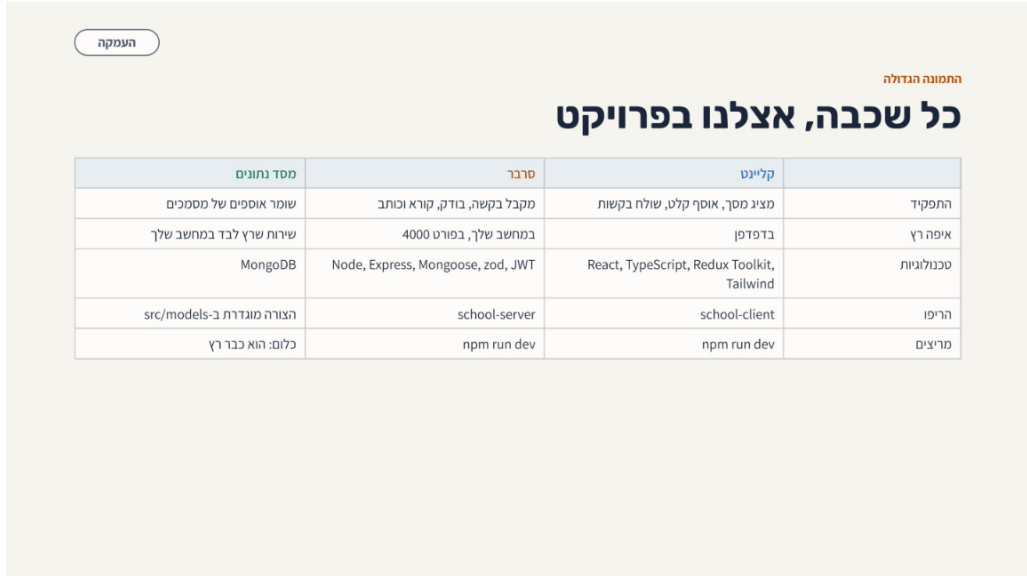


איך מעבירים: לפני השיעור (בבוקר, או שעה לפני) מפרסמות ב-Slack הודעות, הודעה לכל נושא: "כמה את מבינה? הגיבי באימוג'י 1️⃣ עד 5️⃣". כשמגיעות לשיעור כבר יודעות איפה להעמיק, כלל נוסף: אם 3 בנות או יותר סימנו מי שלא הספיקה לענות: מרימות אצבעות בכיתה, לכל נושא. ובשקף הזה רק מציגות לבנות את התוצאות ("ראיתי שגיט ו-PR קיבלו הכי נמוך, אז שם נתעכב"). - 5–4 אצבעות: רק שקף הבסיס איך הופכים את התוצאה להחלטה, בכל נושא: 2–1 בנושא מסוים, מעמיקות בו גם כשהממוצע גבוה. כשעוברים עם העכבר על האימוג'י ב-Slack רואים מי הן. - קליינט, המפה (בסיס | העמקות | בדיקה): - 2–1 אצבעות: בסיס וכל ההעמקות. - 3 אצבעות: בסיס, ההעמקה המומלצת (למטה), ובדיקה. ועצירת הבדיקה. אם הבדיקה עוברת, ממשיכות הלאה. - הכלים: 15 | 16–18 (מומלץ: 17, המסד המקומי. חובה אם מישהי הסתבכה - מבנה הפרויקטים: 10 | 11–13 (מומלץ: 12, מודול בסרבר) | 14 סרבר ו-5–8 | DB: 4 (מומלץ: 6, המסלול של בקשה) | 9 - הפלואו של המסכים שלי: - PR וקוד ריוויו: 19 | 24–25 (מומלץ: שניהם, כי זה מה שעושים מיד אחרי השיעור) | 28 - פקודות גיט: 19 | 20–23, 26–27 (מומלץ: 20 ו-23) | 28 עם seed) | אין בדיקה לשים לב לבנות שמרימות 5 בכל דבר אבל במשימה 29, 31, 32 ו-33 תמיד | 30 | 34. בנושא הזה כדאי לשאול שוב אחרי שקף 30: הרבה בנות ירימו 4 ויגלו בעצירת הבדיקה שהן לא יודעות להסביר. בסוף השיעור (כרטיס היציאה) אפשר לחזור למדחום ולראות אם המספרים עלו. היה להן קשה. בעצירות הבדיקה כדאי לפנות אליהן בשמן, בעדינות.

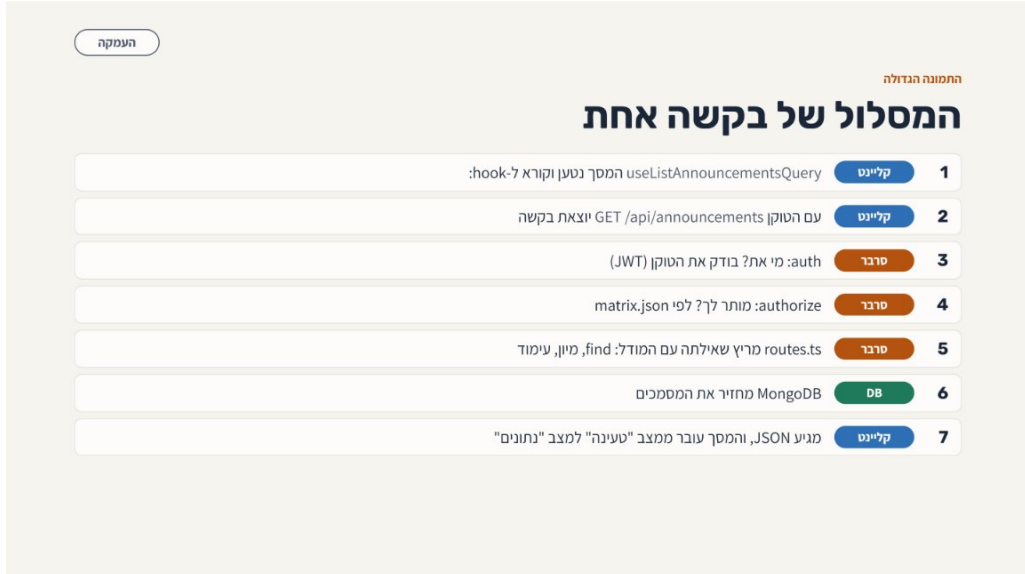


- קליינט = מה שרץ בדפדפן של המשתמשת. הוא יודע להציג ולשלוח בקשות. אי אפשר לסמוך עליו לבד, כי כל אחת להסביר בפשטות: זה השקף הכי חשוב בחלק הזה. לצייר אותו גם על הלוח.
- סרבר = תוכנה שרצה במחשב אחר (אצלכן במחשב, בפורט 4000). מקבל בקשה, בודק מי שלחה אותה (טוקן), אם מותר לה יכולה לפתוח את כלי המפתחים בדפדפן ולשנות אותו.
- מסד נתונים = המקום שבו הנתונים נשמרים גם כשכבים את המחשב. אצלנו MongoDB, שמותקן כשירות במחשב שלכן ועולה לבד. (מטריצת הרשאות), ואם הנתונים תקינים. רק אז ניגש למסד.

החצים: חץ מלא = בקשה. חץ מקווקו = תשובה. בין אחרי השקף הזה: אם במדחום "קליינט, סרבר ו-DB" קיבל 4–5, לקפוץ ישר לעצירת הבדיקה (שקף 10). אחרת, להמשיך בהעמקות 5–9.
שאלה לכיתה: "למה לא לתת לקליינט לגשת ישר ל-MongoDB? היה יותר מהיר." תשובה מצופה: אבטחה. הקליינט לסרבר עוברת בקשת HTTP, והתשובה היא JSON (טקסט במבנה של אובייקט).
כל הרשאה שבודקים רק בקליינט אפשר לעקוף.



השגיאות האדומות בעורך הן הבודק - TypeScript: JavaScript עם טיפוסים. לעבור עמודה עמודה ולשאול "מה כבר נגעתן בו?" מושגים שכדאי לוודא שמבינות: לא לקרוא את הטבלה שורה שורה. Tailwind - Redux Toolkit: עיצוב מקום אחד לנתונים של האפליקציה, ו-RTK Query מנהל את הבקשות לשרת. - React: בונים מסך מרכיבים (קומפוננטות). הראשון שלכן, עוד לפני שמריצים. Mongoose - Express: מגדירים בו את הצורה של מסמך במסד (סכמה), מגדירים בו "כשמגיעה בקשה לכתובת הזו, תריץ את הפונקציה הזו". בעזרת מחלקות קטנות (class) במקום קבצי CSS. הקליינט שולח אותו בכל בקשה, והסרבר יודע ממנו מי - JWT: הטוקן שמקבלים בהתחברות. - zod: בודק שהנתונים שהגיעו בבקשה תקינים. ומקבלים מודל שאפשר לבצע עליו find, create וכו'. שימי לב: גם בקליינט וגם בסרבר הפקודה היא npm run dev, אבל כל אחת - MongoDB: מסד שבו אוסף (collection) מחזיק מסמכים (documents), וכל מסמך דומה לאובייקט JSON. את. זה מבלבל, וכדאי לומר את זה במפורש. מריצה דבר אחר לגמרי.

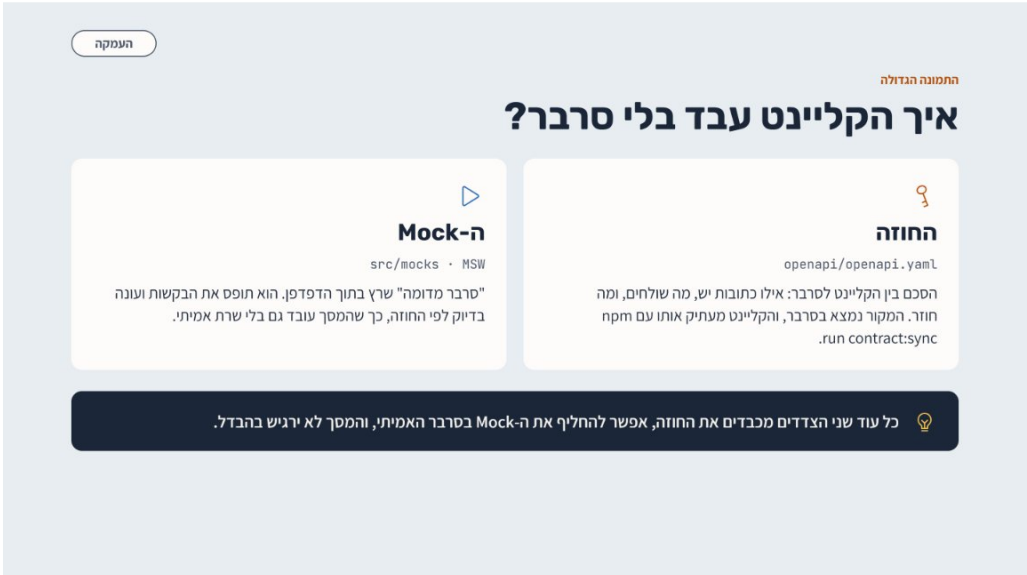


- - משימת הקליינט = שלבים 1, 2 ו-7 (ובמקום שלבים 3–6 היה Mock). להדגיש איפה כל אחת מהמשימות שלהן נמצאת במסלול: לעבור על המסלול עם מסך ההדרכה (הודעות), שכולן מכירות.
- סרבר א' (הבא) = שלב 5: ה-endpoint שקורא מהמסד ומחזיר תשובה לפי החוזה. משימת הסכמות = מה שקורה בשלב 6: הצורה של המסמכים במסד (model.ts) והנתונים שבו (seed.ts).
- "מה קורה אם יש טוקן, אבל מורה מנסה להיכנס למסך של הנהלה?" תשובה: עוצר בשלב 4 - "מה קורה אם אין טוקן?" תשובה: הסרבר עוצר בשלב 3 ומחזיר 401. שאלות לבדוק הבנה תוך כדי: את המסלול הזה מלמדים כאן פעם אחת. בחלק "הפלואו שלי" (שקף 33) כל בת מתכננת את הבקשות של - "איפה במסלול מופיע מצב הטעינה במסך?" תשובה: בין שלב 2 לשלב 7. ומחזיר 403. אם בנות מתבלבלות בין 401 ל-403: 401 = לא יודעים מי את. 403 = יודעים מי את, ואין לך הרשאה. המסך שלה, ושם אפשר להפנות חזרה לשקף הזה.



- method (פעולה): GET = רק קוראים, בקשה בנויה מ: להעמיק בשקף הזה אם במדחום "קליינט, סרבר ו-DB" קיבל 3 ומטה, או אם בנות לא הבינו את המסלול של בקשה. זה הבסיס לסרבר א'.
- כתובת: /api/announcements. אחרי סימן השאלה מגיעים פרמטרים (query): ולא משנים כלום. POST, PATCH ו-DELETE משנים נתונים. בסרבר א' בונים רק GET, ובסרבר ב' את השאר.
- body: רק ב-POST וב-PATCH. הנתונים שנשלחים, בפורמט JSON. - headers: מידע נוסף. החשוב אצלנו הוא Authorization, שבו נמצא הטוקן. page ו-size לעימוד, ובהמשך גם מיון וסינון.
- body: הנתונים, ב-JSON. אצלנו, רשימות חוזרות תמיד באותה צורה: items, total, page ו-size. זו הצורה שבחוזה, וזה מה - קוד סטטוס: מספר שאומר מה קרה (הטבלה). תשובה בנויה מ:

איך רואים את זה בעיניים: בדפדפן, F12, לשונית Network. לרענן את המסך ולהקליק על בקשה. אפשר לראות את ה-method, הכתובת, ה-headers, הסטטוס והתשובה. שהסרבר שלכן יחזיר.
טריק לזכור: 2xx = הצליח. 4xx = הבעיה בבקשה (אצל הקליינט). 5xx = הבעיה בסרבר. כדאי מאוד להדגים, זה פותח להן את הראש.



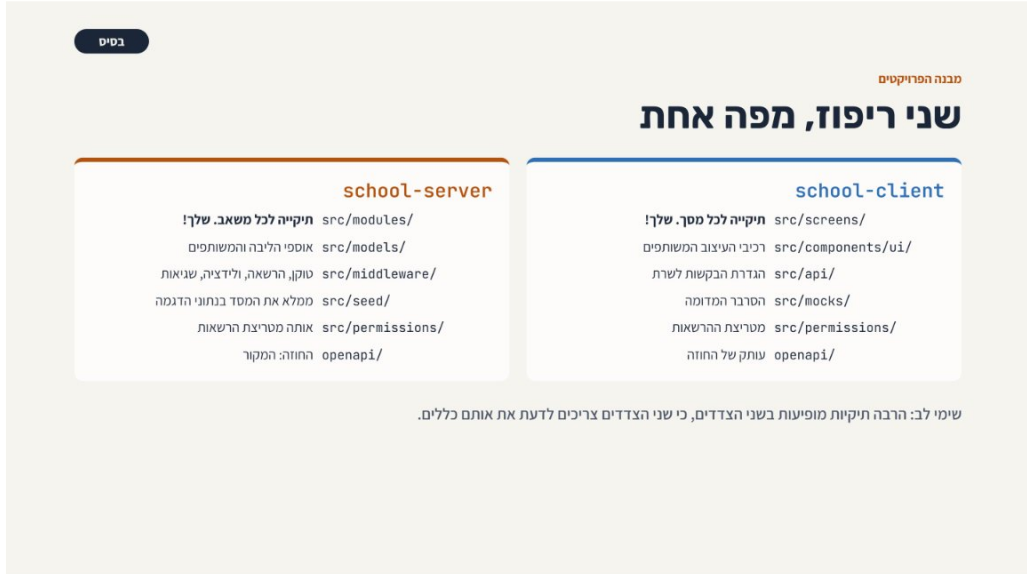
בזכות החוזה, הקליינט והסרבר יכולים להיבנות במקביל, על ידי אנשים שונות, בלי לחכות אחת לשנייה. זה ה"למה" של הסדר שבו עבדנו: קודם קליינט עם Mock, אחר כך סכמות, ועכשיו סרבר. לשאול: "מה יקרה אם בסרבר תחזירו שדה בשם fullName, ובחוזה ובקליינט הוא נקרא name?" תשובה: המסך יציג ריק או יישבר כשנחבר אותו לסרבר האמיתי. לכן בסרבר א' התשובות חייבות השקף הבא (העמקה) מלמד לקרוא endpoint בחוזה, ולפתוח אותו בתוסף Swagger. להזכיר: אף אחת לא עורכת את openapi.yaml בקליינט ביד. רק מעתיקים מהסרבר. להיות זהות לחוזה.



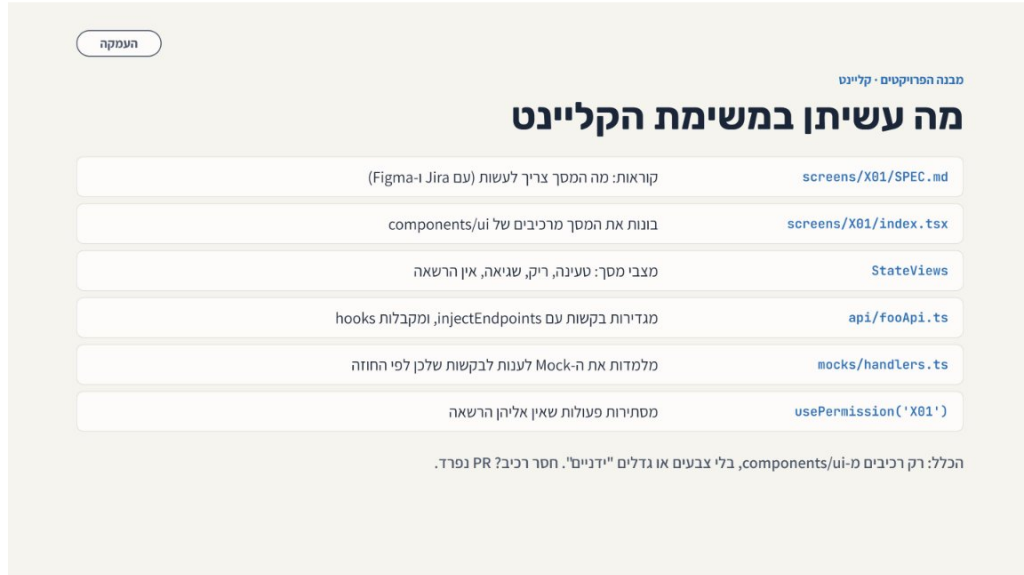
הדוגמה מקוצרת מהחוזה האמיתי של מסך ההודעות. בקובץ עצמו יש עוד השקף הקודם הסביר מה זה חוזה. השקף הזה מלמד לקרוא אותו. בלי זה, "זהים לחוזה" בסרבר א' נשאר משפט ריק. path ו-get: הכתובת והפעולה. זה בדיוק מה שכותבים ב-Postman, וב-routes.ts. לעבור על חמשת המפתחות: שורות (tags, summary, ועוד סטטוסים), אבל אלה החמש שחשובות. x- - parameters: page - security: - ו-size ב-query. זה מה ש-parsePagination קורא בסרבר. screens: הדרך למצוא את ה-endpoints שלך. מחפשות את קוד המסך שלך (Ctrl+F). responses: - לכל סטטוס יש תשובה. ה-$ref מפנה לסכמה בתחתית הקובץ (components/schemas), ושם bearerAuth אומר שצריך טוקן. endpoint בלי security (כמו login) פתוח לכולם. הדגמה חיה, אם יש זמן: להתקין את התוסף OpenAPI (Swagger) Editor, לפתוח את school-server/openapi/openapi.yaml וללחוץ כתובים השדות: items, total, page ו-size. לשאול: "איך Shift+Alt+P. בפעם הראשונה בוחרות Swagger UI. נפתח עמוד עם כל ה-endpoints לפי נושא, ולמטה Schemas. ההוראות המלאות במדריך server-a, בסעיף "לפני שמתחילות". זה מתחבר לשקף "תכנני את הבקשות" תדעי איזה endpoint את בונה בסרבר א'?" תשובה: מחפשות את קוד המסך ב-x-screens, או בטבלה "ה-endpoints בחוזה" בעמוד המסך באפיון. בחלק הפלואו: שם משוות את התכנון שלך לחוזה, ובשביל זה צריך לדעת לקרוא אותו.



1. קליינט. מצב "ריק" הוא מצב התשובות, ומה לומד כל טעות: איך מעבירים: לקרוא כל פריט, וכל הבנות מצביעות יחד: יד ימין = קליינט, יד שמאל = סרבר, שתי ידיים = גם וגם. (או לכתוב על דף.)
2. גם וגם. בקליינט בשביל נוחות (הודעה מיידית בשדה), ובסרבר כי זו מסך (StateViews). מי שענתה "סרבר" עוד לא מבדילה בין הנתונים (הסרבר מחזיר רשימה ריקה) לבין התצוגה שלהם.
3. DB, דרך הסרבר. הסרבר כותב והמסד שומר. מי שענתה "קליינט" הבדיקה שאי אפשר לעקוף (zod ו-israeliId.ts). מי שענתה "רק קליינט" צריכה לשמוע שוב את הכלל: "הקליינט לא אמין".
4. גם וגם. הקליינט מסתיר את הכפתור (usePermission), והסרבר חוסם חושבת שהמידע נשמר בדפדפן. זה קורה כי ב-Mock הנתונים באמת היו בזיכרון של הדפדפן! כדאי להסביר את זה.
5. סרבר (עם שאילתה למסד). לא מביאים 300 שורות לקליינט כדי להציג 20. זה בדיוק מה שתבנו בסרבר א': עימוד, מיון באמת (authorize). זו הטעות המסוכנת ביותר, ולכן כדאי להתעכב עליה.
אם רוב הכיתה טעתה ב-2 וב-4, לחזור לשקף 4 (שלוש השכבות) ולצייר 6. סרבר. הסיסמה שמורה במסד כ-(bcrypt) hash, והסרבר משווה. הקליינט אף פעם לא רואה את הסיסמה השמורה. וסינון.
על הלוח "מה קורה אם מישהי עוקפת את הקליינט".

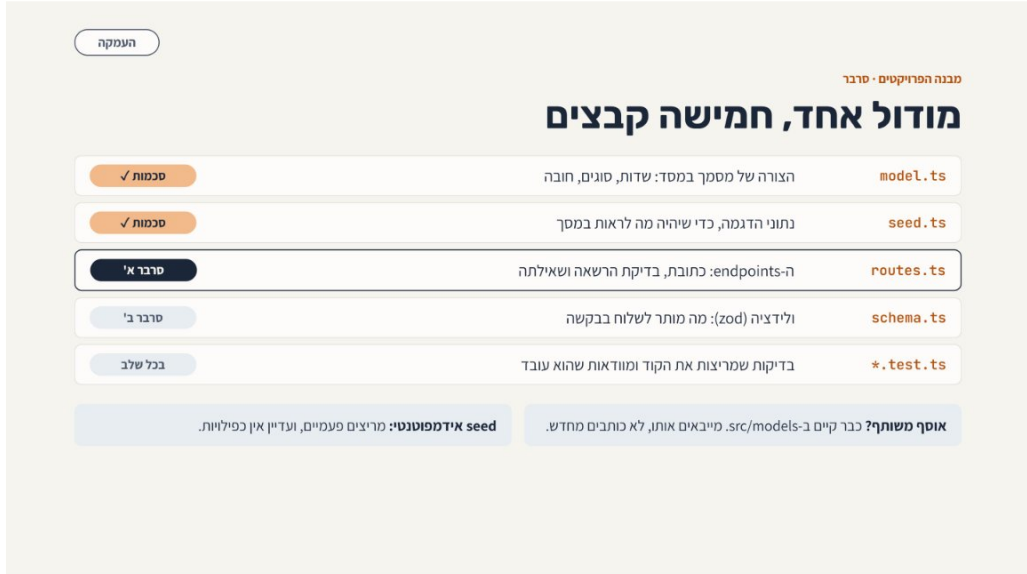


כדאי לפתוח את שני הפרויקטים ב-VS Code זה לצד זה, אחרי השקף הזה: אם במדחום "מבנה הפרויקטים" קיבל 4–5, לקפוץ לעצירת הבדיקה (שקף 15). אחרת, להמשיך בהעמקות 11–13.
המסר: "יש הרבה קבצים, אבל אתן עובדות בעיקר בשתי תיקיות: screens בקליינט ו-modules בסרבר. כל השאר הוא תשתית שמישהי כבר בנתה ולהראות את העץ בזמן שמדברות.
- מטריצת ההרשאות קיימת - בסרבר: משאב = תיקייה ב-modules (למשל announcements). - בקליינט: מסך = תיקייה ב-screens (למשל A01-login). להדגיש את הסימטריה: בשבילכן."
שאלה מהירה: "איפה נמצא הקוד שבודק את הטוקן?" תשובה: - החוזה קיים בשני הצדדים, אבל המקור הוא בסרבר. בשני הצדדים: הקליינט משתמש בה כדי להסתיר, והסרבר כדי לחסום.
src/middleware בסרבר.



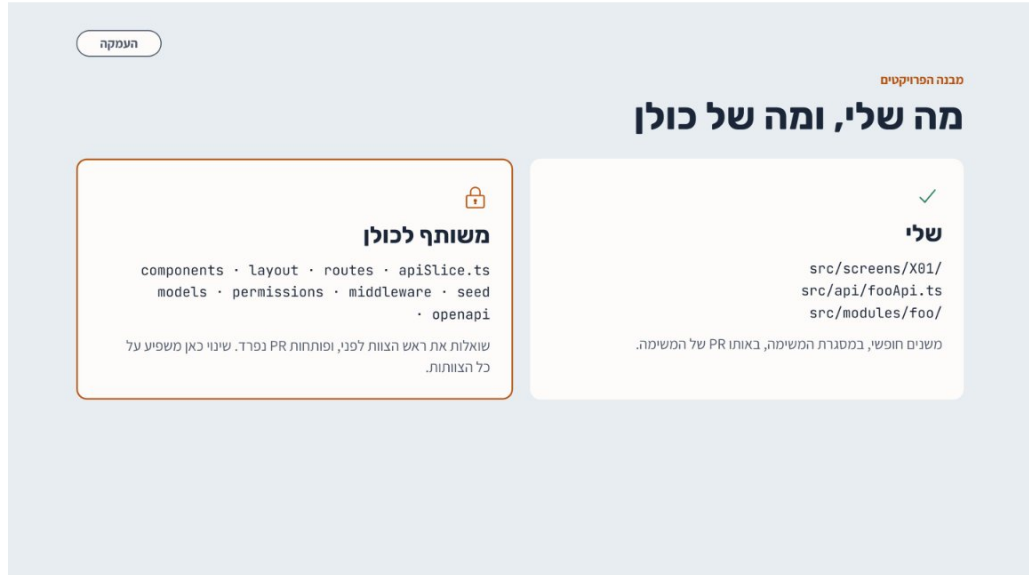
- למה SPEC.md? כדי לא לשאול כל שלב "למה?": X01 הוא שם כללי למסך של כל אחת. לבקש מבת אחת לפתוח את התיקייה של המסך שלה ולהראות לכיתה. זה הופך את השקף לקונקרטי.
- - למה רכיבים מ-components/ui ולא div עם עיצוב? כדי שכל המערכת תיראה אותו דבר, ושינוי אחד ברכיב יתעדכן בכל המסכים. כלל ה-lint אוכף את זה. לנחש. זה מה שבודקים בקוד ריוויו.
- למה api/fooApi.ts נפרד ולא fetch בתוך הקומפוננטה? RTK Query מנהל למה ארבעה מצבי מסך? כי בעולם האמיתי הבקשה לפעמים איטית, לפעמים נכשלת, ולפעמים חוזרת ריקה.

סימן לחוסר הבנה: בת שאומרת "העתקתי מ-announcements ושיניתי שמות". זה - למה Mock? ראינו בשקף 8: כדי לעבוד לפני שיש סרבר. בשבילנו טעינה, שגיאה ומטמון, ונותן hook מוכן. בסדר כנקודת התחלה, אבל לבקש ממנה להסביר שורה אחת ב-fooApi.ts שלה.

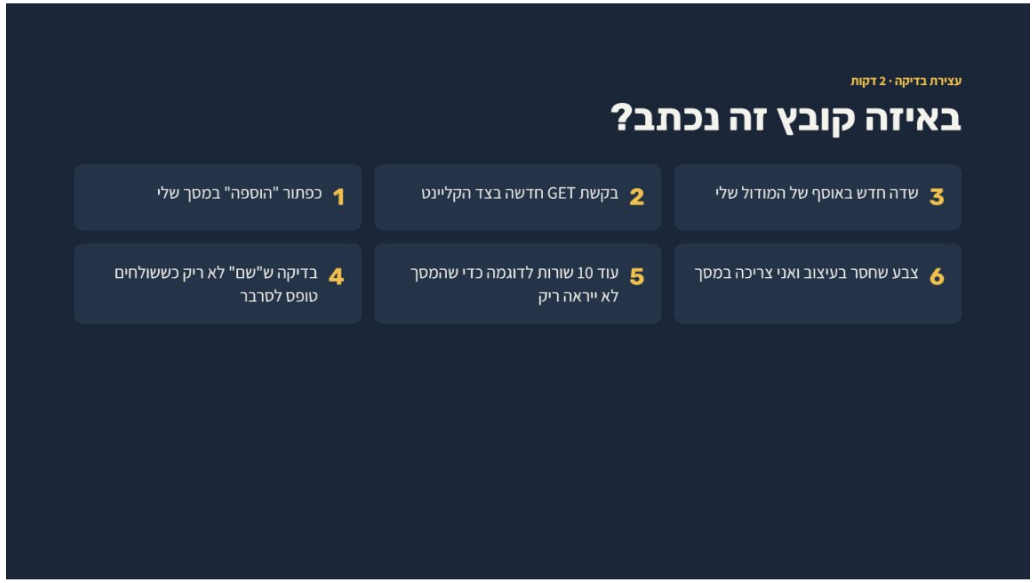


- "סכמה" ו"מודל" ב-Mongoose: הסכמה היא התיאור (אילו שדות, מאיזה סוג, האם חובה). המודל הוא מה שמקבלים מושגים לוודא: החלק הזה מחבר את משימת הסכמות למה שבא אחריה.
- schema.ts זה משהו אחר! שם יש zod, שבודק את מה שמגיע בבקשה. מבלבל כי שניהם נקראים "סכמה". כדאי לומר במפורש: model.ts = מה נשמר במסד. ממנה, ואיתו עושים find ו-create.
- seed אידמפוטנטי: אפשר להריץ אותו שוב ושוב, והתוצאה זהה. אם בנות לא מבינות למה זה חשוב: כל אחת מריצה npm run seed הרבה פעמים, ואם כל schema.ts = מה מותר לשלוח לסרבר.
- למה אסור לייבא ממודול אחר? כדי שכל צמד יוכל לשנות את המודול שלו בלי לשבור לאחרות. מה שמשותף נמצא ב- ריצה מוסיפה עוד 30 תלמידות, תוך שבוע יש 1000 תלמידות כפולות.

המסגרת המודגשת (routes.ts) היא הקובץ המרכזי של סרבר א'. ESLint חוסם את זה אוטומטית. src/models.

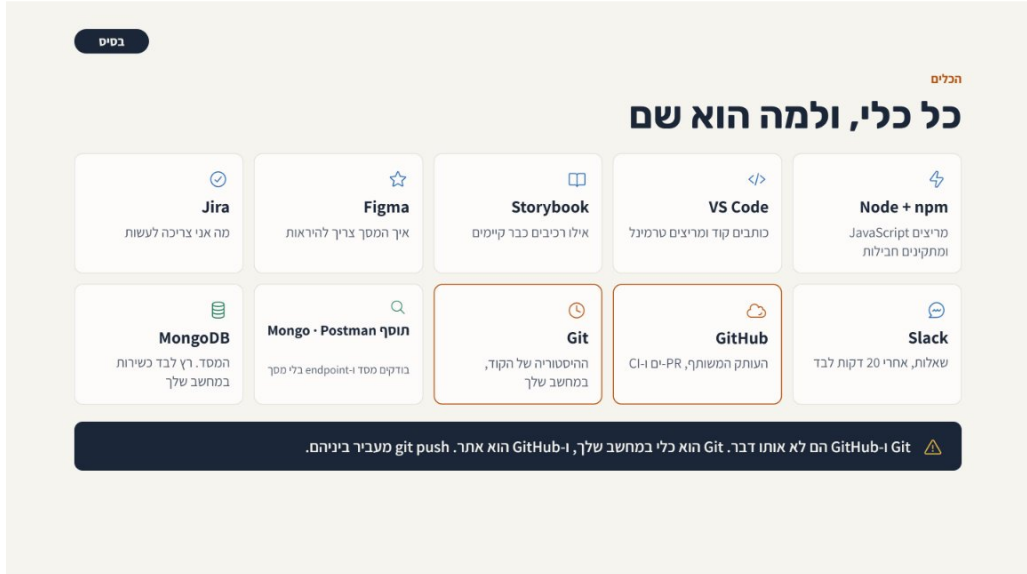


כל PR, שלי או משותף, מאושר על ידי ראש המסר: "לא אסור לגעת בקבצים המשותפים. צריך רק לשאול לפני, ולעשות את זה ב-PR נפרד, כי כל שינוי שם משפיע על המסכים והמודולים של כולן."
סימן לחוסר הבנה: בת שהמסך שלה נראה "לא בדיוק כמו ב-Figma" ושינתה רכיב ב-components/ui בתוך ה- הצוות. ההבדל הוא שבקבצים המשותפים מתאמים מראש, לפני שכותבים קוד.
שאלה לכיתה: "למה שינוי קטן ב-Button יכול להיות מסוכן?" תשובה: הוא מופיע בכל המסכים PR של המסך. הדרך הנכונה: PR נפרד לרכיב (רכיב, בדיקה ו-story), ורק אחר כך ממשיכים במסך.
של כולן.

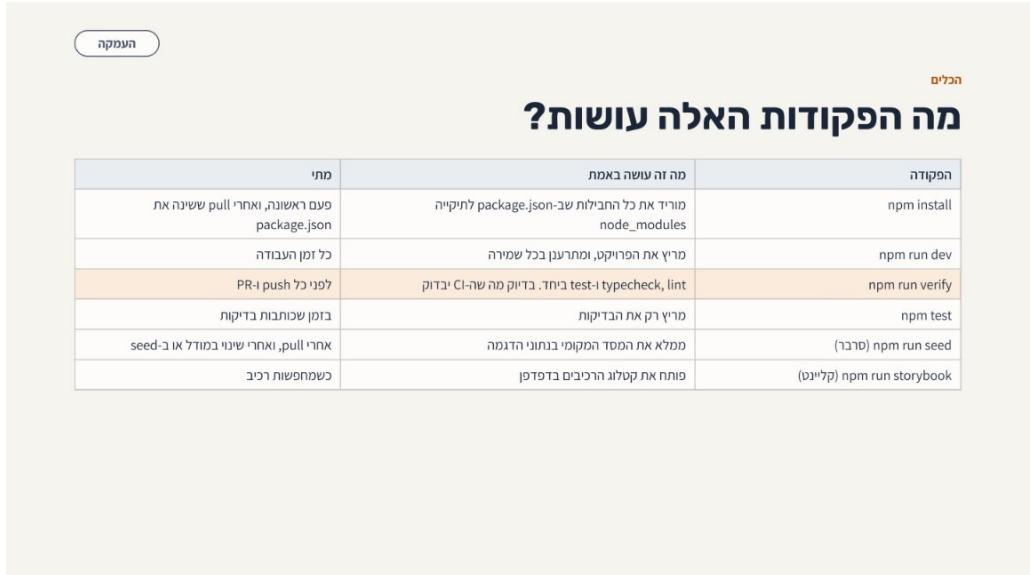


1. school-client: התשובות: לתת לבנות לענות בעל פה, או ב-Slack בבת אחת (כל אחת כותבת 6 תשובות). ב-Slack רואים מי לא בטוחה, גם אם היא שקטה בכיתה.
2. school-client: src/api/fooApi.ts (עם injectEndpoints). לא apiSlice.ts! src/screens/X01/index.tsx
3. school-server: src/modules/foo/model.ts. ואם זה אוסף משותף:
4. school-server: src/modules/foo/schema.ts (zod). אפשר להוסיף גם בדיקה בטופס בקליינט, אבל src/models, ואז זה קובץ משותף: מתאמים עם ראש הצוות לפני, ופותחים PR נפרד.
5. seed.ts של המודול. בקליינט, כל עוד עובדים עם Mock: src/mocks. הבדיקה האמיתית בסרבר.
6. אף אחד מהקבצים שלי. זה שינוי משותף (tailwind.config / טוקנים): שואלים את בסרבר:

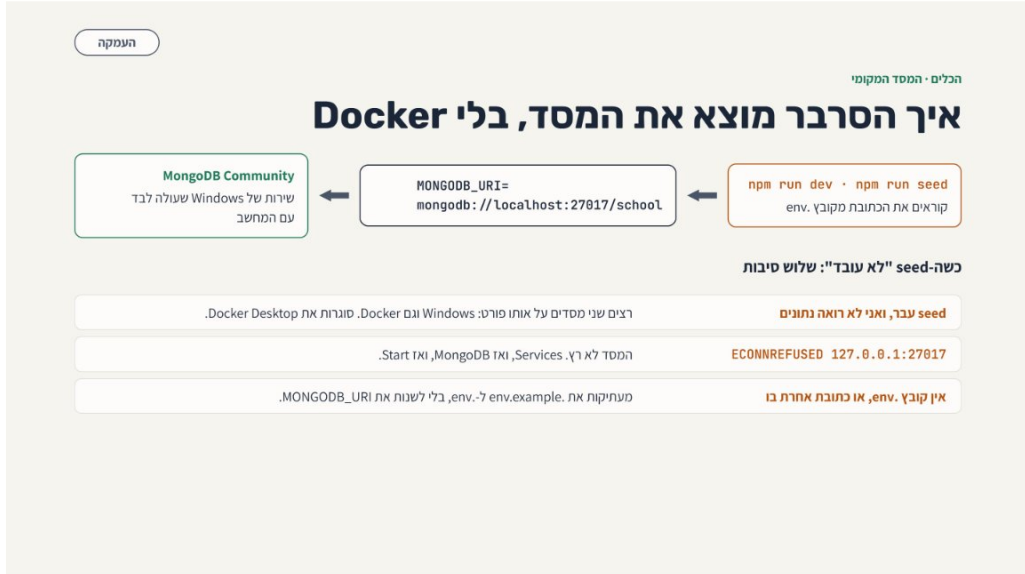
אם רוב הטעויות היו ב-3 או ב-4: לחזור לשקף 13, ולהסביר שוב את ההבדל בין model.ts ל-schema.ts. ראש הצוות ופותחים PR נפרד.



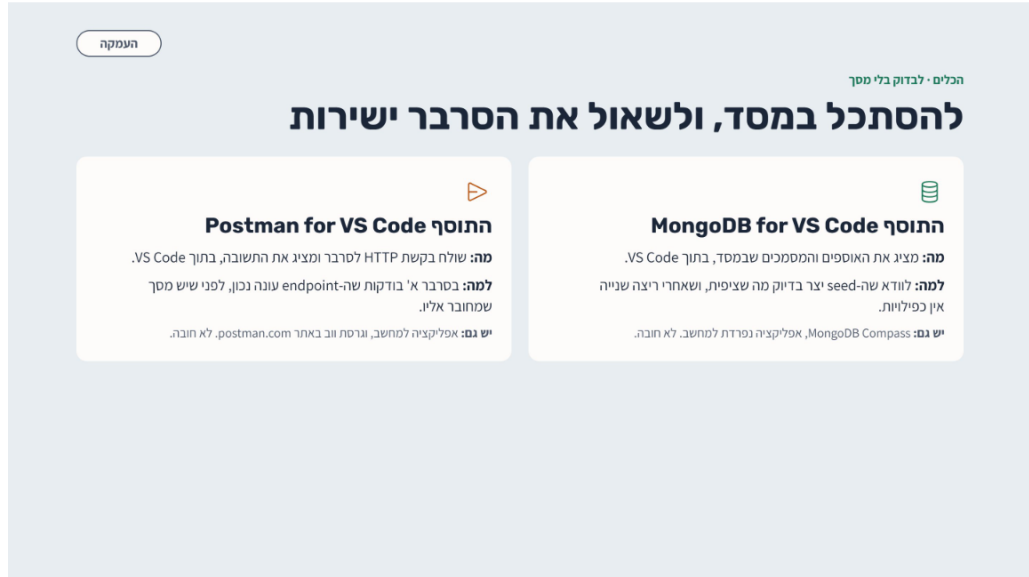
לסדר את הכלים לפי זרימת העבודה, מימין לשמאל ומלמעלה למטה: קודם מבינות את המשימה (Jira, Figma, Storybook), אחר כך כותבות ומריצות (VS Code, Node, MongoDB, התוסף ו-MongoDB: מותקן פעם אחת כשירות של Windows, ועולה לבד עם המחשב. אין צורך להריץ פקודה כדי שהוא יעבוד, ואנחנו לא משתמשות ב- Postman), ובסוף משתפות (Git, GitHub, Slack).
תוסף Mongo ו-Postman: שני כלים לבדוק דברים בלי מסך. התוסף מראה מה יש במסד (למשל, אחרי seed). Postman שולח בקשה לסרבר ומציג את התשובה. נצטרך את שניהם Docker.
ההבדל בין Git ל-GitHub הוא הבלבול הכי נפוץ, לשאול: "אם המסך שלי לא תואם ל-Figma, איפה אני בודקת אם יש כבר רכיב מתאים?" תשובה: Storybook (npm run storybook). בסרבר א'.
- העמקות בחלק הזה, לפי הצורך: והוא הבסיס לכל החלק של גיט. אם יש ספק, לשאול: "אם אין אינטרנט, אפשר לעשות commit?" תשובה: כן! commit הוא מקומי. רק push צריך אינטרנט.
- תוסף Mongo ו-Postman (19): אפשר - המסד המקומי (18): חובה אם מישהי הסתבכה עם seed או התקינה Docker. פקודות ה-(17) npm: אם בנות מריצות פקודות בלי לדעת מה הן עושות. לדלג היום ולהציג בפתיחה של סרבר א'.



המסר: "X npm run זה לא קסם. זה רק קיצור לפקודה שכתובה ב-package.json, בחלק "scripts." לפתוח את package.json ולהראות את זה. למשל verify הוא בעצם שלוש פקודות ברצף. test = מריץ את כל - lint = ESLint בודק כללים: למשל, שאין צבע "ידני" או ייבוא ממודול אחר. - typecheck = TypeScript בודק שאין שגיאות טיפוסים. להתעכב על verify (השורה הצבועה): seed: עובד רק כשהשירות MongoDB ה-CI ב-GitHub מריץ את אותה פקודה על כל PR. אם verify עבר אצלך, כמעט בטוח שגם ה-CI יעבור. אם לא הרצת אותו, ה-PR יקבל X אדום. הבדיקות. שאלה: "משכתי שינויים מ-dev, ופתאום npm run dev נכשל עם Cannot find module. מה עושים?" תשובה: רץ במחשב (ראו בשקף הבא). לא צריך Docker ולא צריך להריץ שום פקודה לפני. npm install, כי מישהי הוסיפה חבילה.



איך זה עובד: הסרבר שלנו לא "יודע" איפה המסד. הוא קורא את הכתובת מהמשתנה MONGODB_URI זה שקף חובה לכל מי שהסתבכה עם seed. אם יש אפילו בת אחת כזו, כדאי להציג אותו. בקובץ .env. ברירת המחדל היא localhost:27017, כלומר "המחשב שלי, פורט 27017". מי שמקשיב בפורט הזה הוא MongoDB Community Server. לפי ה-README, מתקינים אותו פעם אחת Docker הוא רק עוד דרך להריץ את אותו MongoDB. עם האפשרות Install MongoD as a Service, ומאז הוא עולה לבד בכל הפעלה של המחשב. לא צריך להריץ שום פקודה כדי שהמסד יעבוד. מה קרה לבנות שהתקינו Docker: היו להן שני שרתי MongoDB שמנסים להקשיב לאותו פורט 27017, אחד של Windows ואחד בתוך Docker. ה-seed כתב לאחד, והן אנחנו לא משתמשות בו. הפתרון לבנות האלה: לסגור את Docker Desktop לגמרי (קליק ימני על האייקון ליד השעון, Quit), לוודא הסתכלו (או שהסרבר התחבר) לשני. לכן נראה כאילו ה-seed "לא ידע על מה לרוץ". איך בודקים שהשירות רץ: Win+R, לכתוב services.msc, ולחפש MongoDB Server. הסטטוס צריך להיות שהשירות MongoDB ב-Windows במצב Running, ולהריץ שוב npm run seed. להעתקה ב-env. (Git Bash: cp .env.example .env) PowerShell: Copy-Item .env.example .env. Running.



- המסד: האם הנתונים ההיגיון: עד עכשיו בדקתן הכול דרך המסך. אבל בסרבר א' עוד אין מסך שמחובר לסרבר האמיתי, כי הקליינט עדיין עובד עם Mock. לכן צריך דרך לבדוק כל שכבה בנפרד: התוסף: Ctrl+Shift+X, מחפשות MongoDB for VS Code, מתקינות. בסרגל - הסרבר: האם ה-endpoint מחזיר את התשובה הנכונה? ב-Postman. שם? בתוסף MongoDB for VS Code. השמאלי לוחצות על העלה הירוק, ואז Add Connection, ואז Connect with Connection String, ומדביקות mongodb://localhost:27017. (כתוב גם ב-README של השרת.) בפנים, המסד כותבות את הבקשות ידנית, לפי החוזה: method, כתובת, ובלשונית Authorization טוקן מסוג Bearer. Postman: תוסף ב-VS Code. כל בת נרשמת לחשבון חינמי. school, ובו כל האוספים. למה ידני ולא ייבוא: כשכותבים את הבקשה בעצמן, רואים בדיוק מה קודם שולחות בקשת התחברות (POST /auth/login) ומעתיקות את הטוקן מהתשובה. ההוראות המלאות במדריך server-a. שני הכלים הם תוספים ב-VS Code, כדי שהכול יהיה בחלון אחד. לשניהם יש גם גרסה נפרדת: MongoDB Compass, והאפליקציה או גרסת הווב של Postman. בתוסף הקליינט שולח לסרבר. לשים לב: גרסת הווב של Postman לא מגיעה לבד ל-localhost. צריך להתקין לשם כך ובאפליקציה מתחברים עם אותו חשבון Postman, כך שבקשות שנשמרו בתוסף מופיעות גם באפליקציה. אם לא הספקתן: אין צורך להתעכב כאן כדאי להדגים בכיתה: בקשה בלי טוקן מחזירה 401. זה מחבר לשקף המסלול של בקשה. את Postman Desktop Agent. לכן אצלנו עובדים עם התוסף. היום. מספיק שידעו ששני הכלים קיימים, ושנשתמש בהם בסרבר א'.



לספר את זה כסיפור, מימין לשמאל: "פותחים ענף משלך, כדי לא להפריע לאחרות. שומרים את זה הבסיס של כל החלק. אם הבנות יוצאות מהשיעור רק עם השקף הזה, הן כבר במקום טוב. העבודה בשמירות קטנות (commit). מעלים אותן ל-(push) GitHub. פותחים PR, שזה בקשה למזג. ראש הצוות קוראת ומאשרת, או מבקשת תיקונים. ואז ממזגים ל-dev, וכולן מקבלות את - 3 אצבעות: ארבעת המקומות (21), - 4–5 אצבעות: לדלג ישר למחזור העבודה המלא (שקף 24) ולעצירת הבדיקה (שקף 29). אחרי זה, לפי המדחום של "פקודות גיט" ו-"PR וקוד ריוויו": הקוד." שאלה מהירה לפני שממשיכות: "מה ההבדל בין - 1–2 אצבעות: לעבור על כל ההעמקות לפי הסדר. אם צריך, לקצר בחלקים אחרים. מחזור העבודה (24), (25) PR, ריוויו (26), ואז הבדיקה. commit ל-push?" אם יש היסוס, להציג את שקף 21 בכל מקרה.

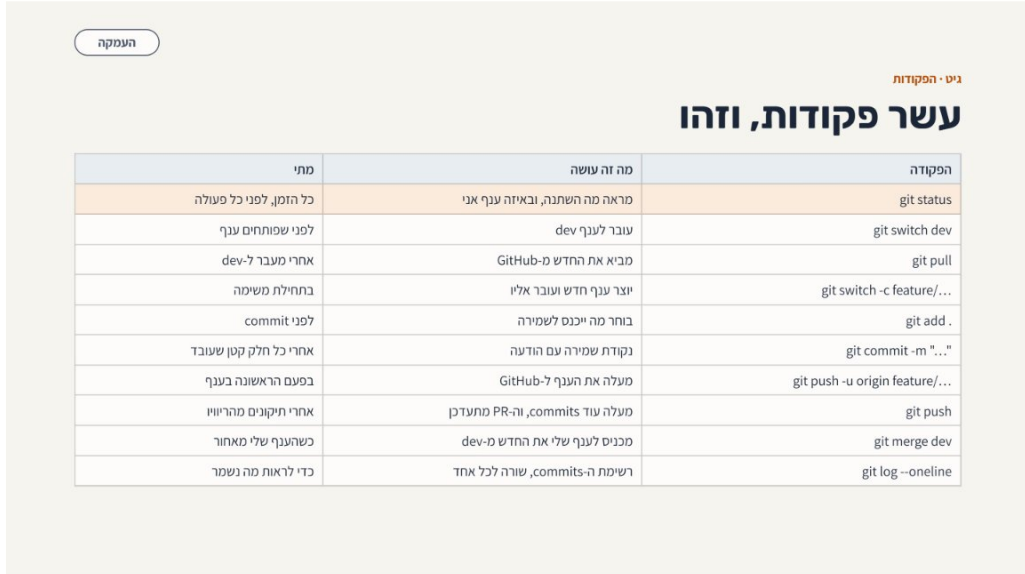


1. עורכת קובץ ושומרת (Ctrl+S). גיט רואה שהקובץ השתנה, אבל עוד לא שמר כלום. להסביר מימין לשמאל: זה השקף שהכי חשוב שיבינו. בלי התמונה הזו, הפקודות הן רק מילים לשנן.
2. git add
3. git commit = נקודת שמירה עם הודעה. נשמרת רק במחשב שלך! אם המחשב נשרף, היא אבדה. = "את הקובץ הזה אני רוצה בשמירה הבאה". אפשר לבחור רק חלק מהקבצים.
4. git push = הכיוון ההפוך. מביא commits שאחרות העלו. מעלה את ה-commits ל-GitHub. רק מעכשיו אחרות יכולות לראות אותם.

להדגים חי בטרמינל (2 דקות): לשנות שורה, git status git pull
- "שמרתי קובץ ב-VS Code. שומר. commit בוחר, commit?" add-ל add ההבדל בין "מה
- "עשיתי commit. החברה שלי רואה את זה?" לא, רק אחרי push. שאלות אבחון: (אדום) git add, git status (ירוק), git commit, git status (נקי), git log --oneline.
- "commit זה Code. שומר commit בוחר, commit?" add-ל add ההבדל בין "מה
- "עשיתי commit. החברה שלי רואה את זה?" לא.



1. dev מעודכן. לעבור על התרשים: ענף = קו פיתוח נפרד. אפשר לעבוד בו בלי להפריע לאחרות, ובלי שאחרות יפריעו לך. הזמן בתרשים זורם מימין לשמאל, כמו הקריאה. כל עיגול הוא commit.
3. מיזוג חזרה ל-dev קורה רק 2. עושות כמה commits בענף שלך. בינתיים dev ממשיך להתקדם בלעדייך (העיגול באמצע) כי אחרות ממזגות. פותחות ממנו ענף feature משלך (git switch -c).
למה צריך גם dev וגם stage? ב-dev יכולים להיות דברים שעוד לא נבדקו עד הסוף. stage הוא מה שמראים, אז הוא 4. פעם בשבוע, ביום רביעי, dev ממוזג ל-stage. דרך PR (העיגול החלול).
שם הענף: feature/PEDB-12- הענפים dev ו-stage מוגנים ב-GitHub: אי אפשר לעשות אליהם push ישיר. אם בת מנסה git push כשהיא על dev, היא תקבל שגיאה. וזה טוב! צריך להיות יציב.
שאלה: "בזמן שעבדתי, חברה מיזגה ל-dev. הענף שלי יודע על זה?" לא! צריך להביא את השינויים בעצמך S01-serverA. מפתח ה-Jira בשם הענף מחבר את הענף למשימה, ו-Jira מציג אותו.
(שקף 27).



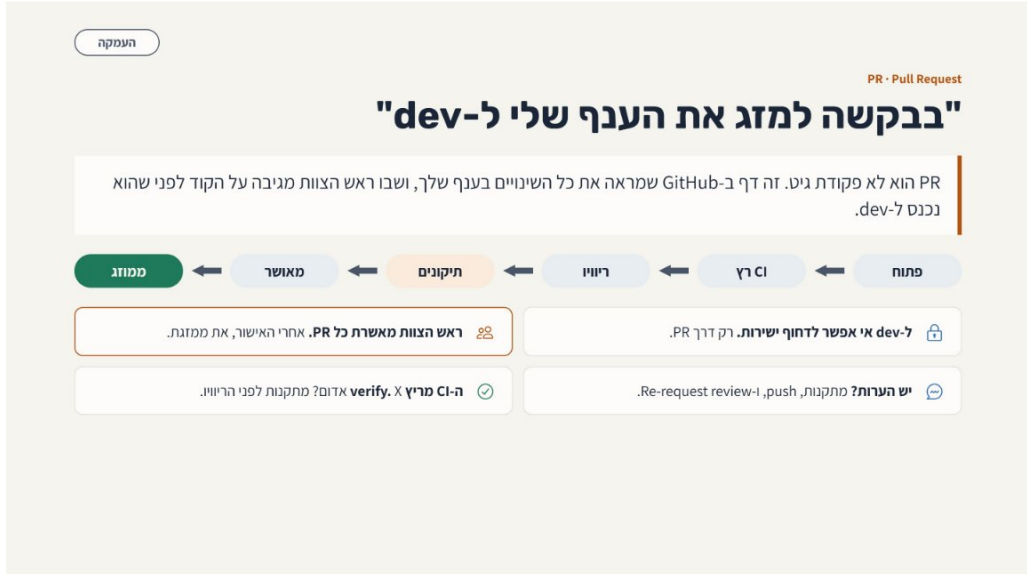
- git status: הכלל הכי חשוב. "לפני כל פקודה, git status". הוא גם אומר באיזה דגשים: לא לשנן את הטבלה. להראות שכל פקודה מתאימה לחץ בשקף 21 (ארבעת המקומות) או לענף בשקף 22.
- git switch - u- - : הפקודה החדשה. בהרבה מדריכים כתוב git checkout, וזה אותו דבר. git checkout -b זהה ל-git switch -c. ענף את נמצאת. רוב הטעויות קורות כשעובדים בענף הלא נכון.
- git add . (עם נקודה) = כל - origin: השם של העותק ב-GitHub. כשכתוב origin, הכוונה ל-GitHub. ב-push הראשון: מחבר את הענף המקומי לענף ב-GitHub. אחרי זה מספיק git push.
- - הודעת commit: באנגלית, קצרה, עם מזהה המסך. למשל "S01: add list endpoint with paging". הקבצים שהשתנו. לפני כן להריץ git status ולוודא שאין שם קבצים שלא שייכים למשימה.
- שאלה: "מה ההבדל בין git pull ל-git merge dev git merge: חשוב שה-dev המקומי יהיה מעודכן קודם (switch dev, pull, ואז switch בחזרה לענף שלך). יש גם קיצור: git pull origin dev. pull "?dev מביא מ-GitHub אל הענף שאני עליו. merge dev מערבב את dev לתוך הענף שאני עליו.



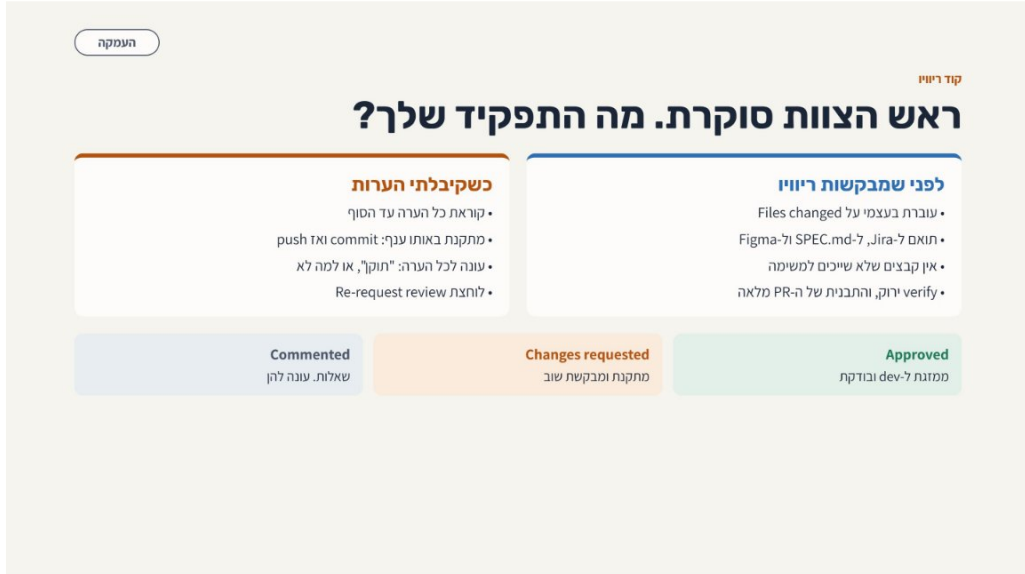
לעבור צעד צעד ולחבר לשקפים הקודמים: צעדים 1–2 = ענפים (שקף 22). צעד 3 = ארבעת המקומות (שקף 21). צעד זה ה"מתכון" כולו. כדאי להדפיס את השקף הזה ולתלות אותו ליד כל מחשב.

- צעד 1 נשכח: פותחות ענף כשהן עדיין על הענף של הנקודות שבהן בנות נתקעות בדרך כלל: גם אם דילגת על כל שאר ההעמקות בגיט, מומלץ להציג את השקף הזה. 4 = verify (שקף 17).
- צעד 3: עושות commit אחד ענק בסוף. עדיף כמה commits קטנים, כל אחד עם הודעה המשימה הקודמת. התוצאה: ה-PR החדש מכיל גם את כל הקוד הישן. לכן תמיד מתחילים מ-dev.
- צעד - צעד 9: פותחות PR חדש אחרי כל תיקון. לא צריך! push לאותו ענף מעדכן את ה-PR הקיים לבד. - צעד 6: בוחרות base לא נכון. ה-base צריך להיות dev (לא main ולא stage). ברורה.

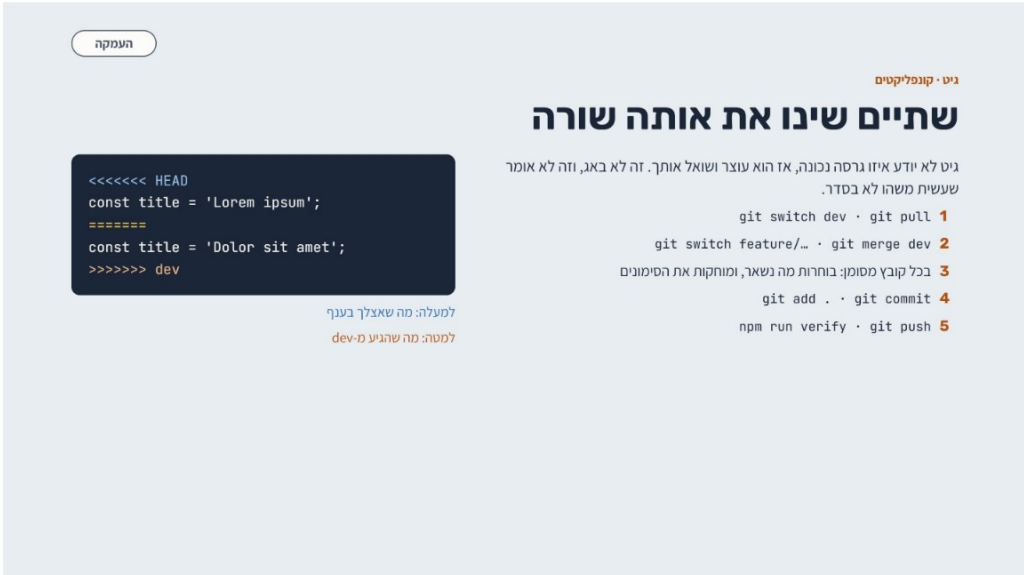
שאלה: "למה בצעד 1 צריך pull? פתחתי את dev רק אתמול." תשובה: מאז אחרות מיזגו. ענף 10: אחרי ה-Merge ממשיכות לעבוד על הענף הישן. ענף שמוזג נגמר. מוחקות אותו ופותחות חדש. שנפתח מ-dev ישן יביא לקונפליקטים.



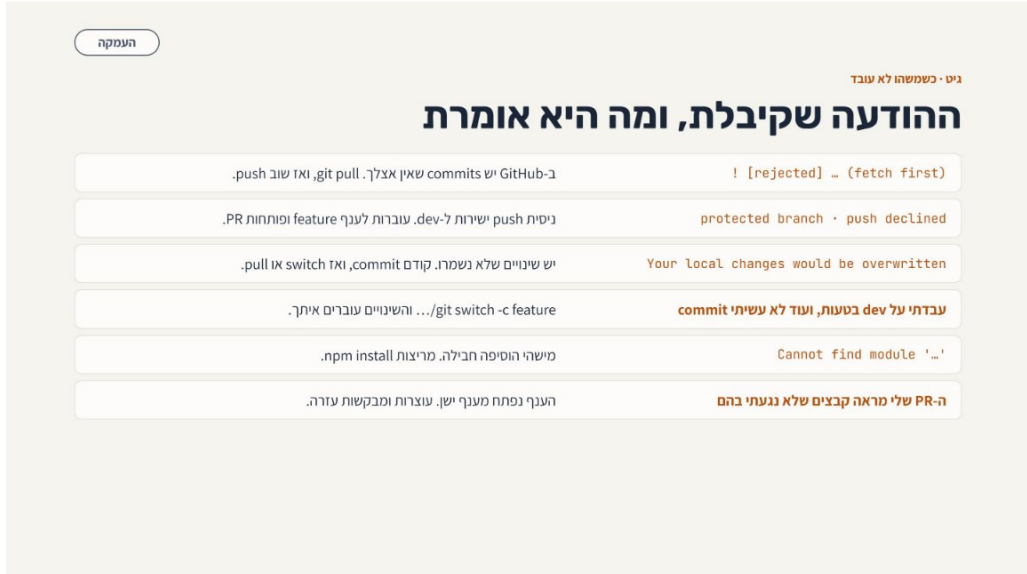
Conversation - התיאור (מהתבנית) וכל מה יש ב-PR: הבלבול הכי נפוץ: בנות חושבות ש-PR הוא פקודה בטרמינל. להדגיש: git push מעלה את הענף. את ה-PR פותחים בדפדפן, ב-GitHub. התהליך - Files changed: ה-diff, כלומר מה נוסף (ירוק) ומה נמחק (אדום). כאן ראש הצוות כותבת הערות על שורות. - Checks: התוצאה של ה-CI. - Commits: כל ה-commits בענף. ההערות. אצלנו: ראש הצוות סוקרת ומאשרת כל PR. אחרי Approve, הבת עצמה לוחצת Merge. אם יש Request changes, הבת מתקנת באותו ענף, עושה push, ולוחצת Re-request review ליד השם "למה ה-PR צריך להיות קטן (עד 400 שורות)?" כי "הכפתור Merge אפור": בדרך כלל אין עדיין אישור. לבדוק גם שאין קונפליקט עם dev. של ראש הצוות (בצד ימין של ה-PR, תחת Reviewers). להראות PR אמיתי של אחת הבנות מהמשימה האחרונה (אם היא מסכימה) ולעבור על ארבע הלשוניות. ההערות אף אחת לא באמת קוראת 2000 שורות. PR קטן מקבל ריוויו טוב יותר ומהר יותר. של ראש הצוות כבר שם, וזה בדיוק מה שהן יעשו אחרי השיעור.



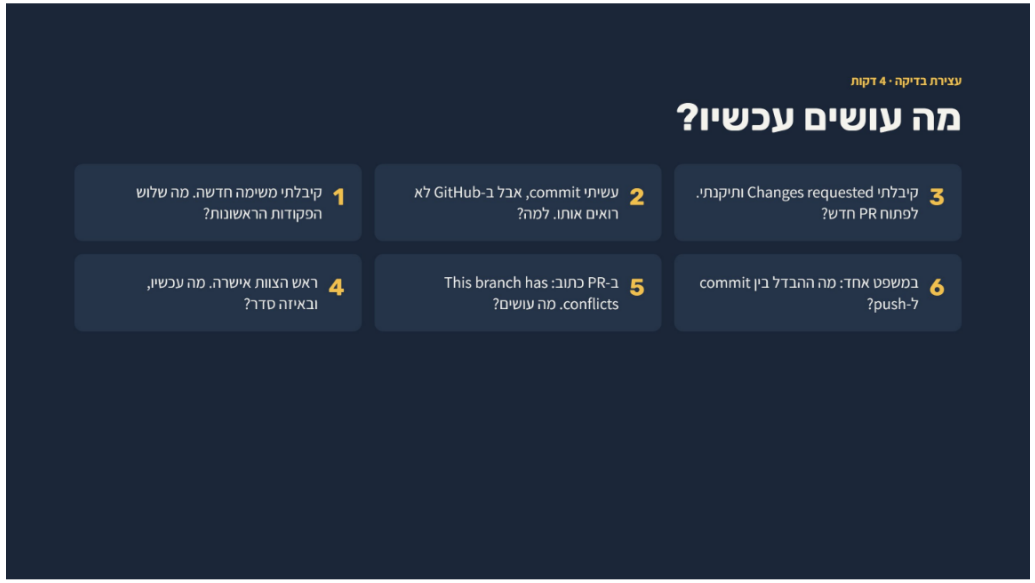
ריוויו עצמי (צד ימין): לפני שמבקשות ריוויו, כל אחת פותחת את ה-PR שלה ב- אצלנו ראש הצוות היא הסוקרת של כל PR. אבל לבנות יש תפקיד בריוויו: להכין את ה-PR טוב, ולטפל בהערות נכון. GitHub, בלשונית Files changed, וקוראת אותו כאילו היא הסוקרת. ככה תופסים לבד קבצים שנכנסו בטעות, console.log שנשכח, או שורות שלא קשורות למשימה. זה חוסך סבב הערות שלם. - עונים לכל הערה, גם אם רק "תוקן". ככה - לא פותחים PR חדש. push לאותו ענף, וה-PR מתעדכן. - הערה על הקוד היא לא הערה עלייך. היא חלק מהלמידה. כשמקבלות הערות (צד שמאל): Re-request review - בצד ימין של ה-PR, תחת Reviewers, יש חץ מעגלי ליד השם - לא מסכימה עם הערה? כותבת למה, בנימוס. שאלה היא גם תשובה טובה. ראש הצוות יודעת מה לבדוק שוב. שלושת הסטטוסים בתחתית: זה בדיוק מה שהבנות יראו ב-PR שלהן מיד אחרי השיעור. כדאי לשאול: "מה כתוב אצלך?" של ראש הצוות. בלי זה, היא לא יודעת שסיימת.



מתי להרגיע: קונפליקט הוא חלק רגיל מעבודה בצוות. אחרי השיעור, כשנמזג כמה PR-ים ל-dev אחד אחרי השני, כמעט בטוח שנקבל אחד. זו הזדמנות מצוינת לפתור אותו ביחד, על המסך. ב-GitHub רואים בתחתית ה-PR: קונפליקט קורה? כששתיים שינו את אותן שורות באותו קובץ (או שאחת מחקה קובץ ששנייה שינתה). שינויים בקבצים שונים, או בשורות רחוקות, גיט ממזג לבד. פותרים במחשב (שלבים 1–5). VS Code מציג כפתורים מעל הקונפליקט: Accept Current (שלך), Accept Incoming (מ-dev), Accept Both. לפעמים התשובה הנכונה היא שילוב ידני של שתיהן. להראות ש- חשוב: אחרי פתרון, להריץ verify! קוד שנפתר לא נכון יכול להתקמפל ועדיין להיות שבור. אם בת נבהלת באמצע: git merge --abort מחזיר הכול למצב שלפני ה-merge. אפשר לעצור ולבקש עזרה. HEAD פירושו "איפה שאני נמצאת עכשיו", כלומר הענף שלך. הכפתור Merge לא זמין. "This branch has conflicts that must be resolved"

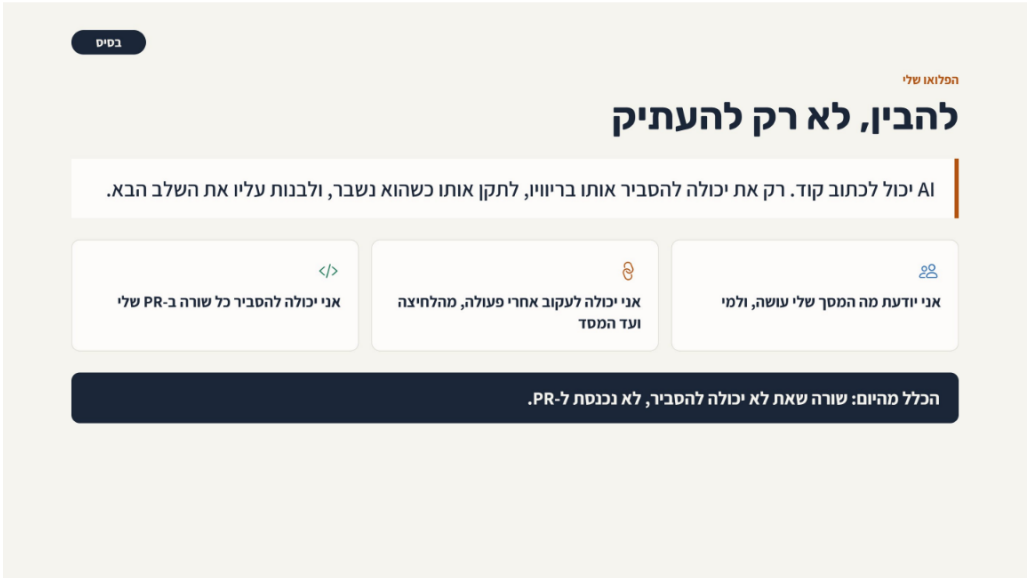


עוד כלל: לפני שמנסים פקודה הכלל הכי חשוב: לקרוא את הודעת השגיאה. רוב הבנות רואות טקסט אדום באנגלית וישר עוצרות. ברוב המקרים גיט אפילו כותב בהודעה מה לעשות ("hint: ..."). להסביר את השורה השלישית: גיט לא מוכן לעבור ענף או למשוך שינויים, אם זה "מתקנת" שמצאתן ברשת (במיוחד עם --force או reset --hard), שואלות. אלה פקודות שיכולות למחוק עבודה. השורה הרביעית: שינויים שלא נשמרו ב-commit "הולכים איתך" ידרוס קבצים ששינית ולא שמרת. הוא מגן עלייך. אפשר לעשות commit, או git stash (שם בצד זמנית) ואחר כך git stash pop. השורה האחרונה: זו הבעיה שהכי קשה לתקן לבד. עדיף שראש הצוות תתקן, ותוך כדי כשעוברים ענף. לכן אם עבדת על dev ועוד לא עשית commit, פשוט פותחים ענף חדש ועושים commit שם. שאלה: "קיבלתי שגיאה ב-push. מה הפקודה הראשונה שאני מריצה?" תשובה: git status, וקריאת ההודעה. יסבירו.

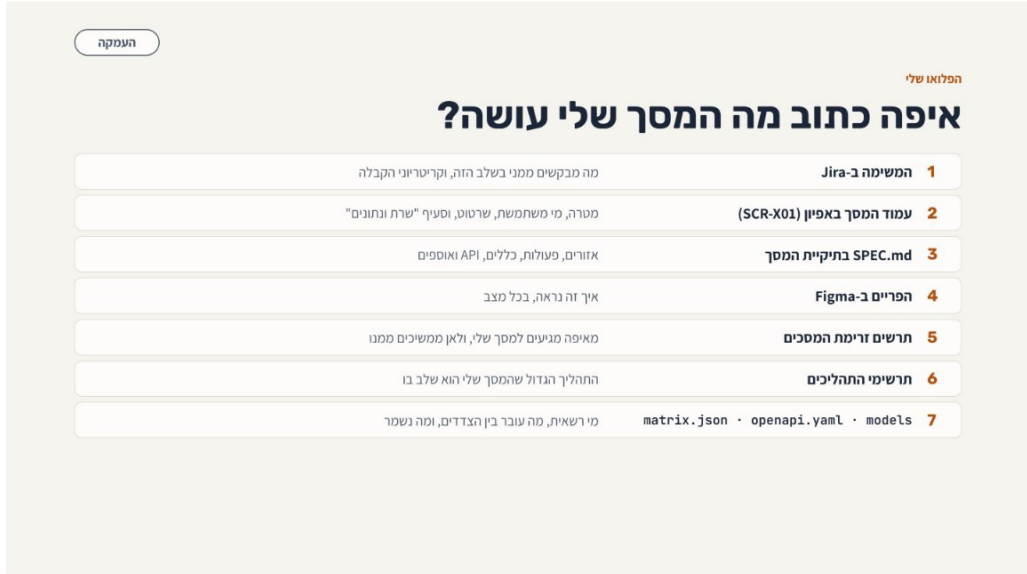


1. התשובות, ומה לומדים מטעות: זו העצירה החשובה ביותר בשיעור. לתת דקה לכתוב תשובות על דף או ב-Slack, ורק אחר כך לעבור יחד. ככה רואים כל אחת, ולא רק את מי שעונה מהר. git
2. לא עשיתי commit. push נשמר רק ב-dev ב-GitHub מתקדם בלי שהיא רואה. לחזור לשקף 22 (ענפים). switch dev, git pull, git switch -c feature/... מי ששכחה pull, לא מבינה עדיין ש-
3. לא! add, commit ו-push לאותו ענף, וה-PR מתעדכן לבד. ואז Re-request במחשב. מי שטעתה כאן, לא הפנימה את ארבעת המקומות. לחזור לשקף 21. זו הטעות החשובה ביותר לתפוס.
4. Merge ב-GitHub. אחר כך במחשב: git switch dev, git pull, npm install, ובודקות review. מי שטעתה, חושבת ש-PR הוא "שליחה" חד-פעמית ולא דף שמתעדכן. לחזור לשקפים 25–26.
5. במחשב: switch dev, pull, switch לענף שלי, merge dev, פותרות add, commit, verify, push. שהכול עובד. מי שעונה "git push ל-dev" עוד לא הבינה ש-dev מתעדכן רק דרך PR.

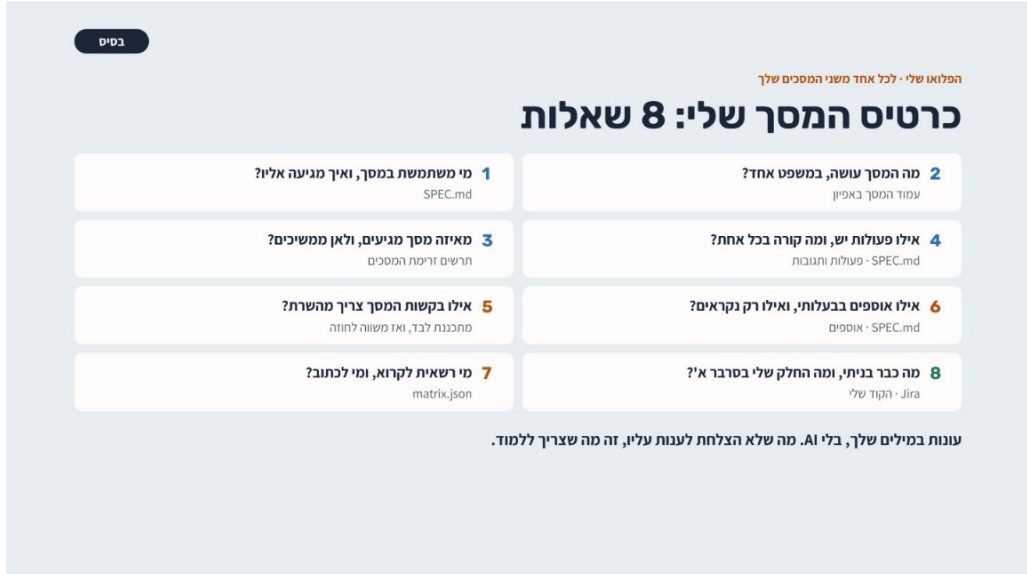
שאלות 3 ו-4 הן בדיוק מה שהבנות יעשו מיד אחרי השיעור. אם הן נכשלו בהן, להציג את שקף 36 (מה commit = שמירה במחשב שלי. push = העלאה ל-GitHub, כדי שאחרות יראו. שקף 27. אם יותר ממחצית הכיתה טעתה ב-2 או ב-3: לוותר על סרבר א' (שקף 37) ולהשתמש בזמן לתרגול חי בטרמינל. עכשיו) לאט, ולהדגים על PR אמיתי.



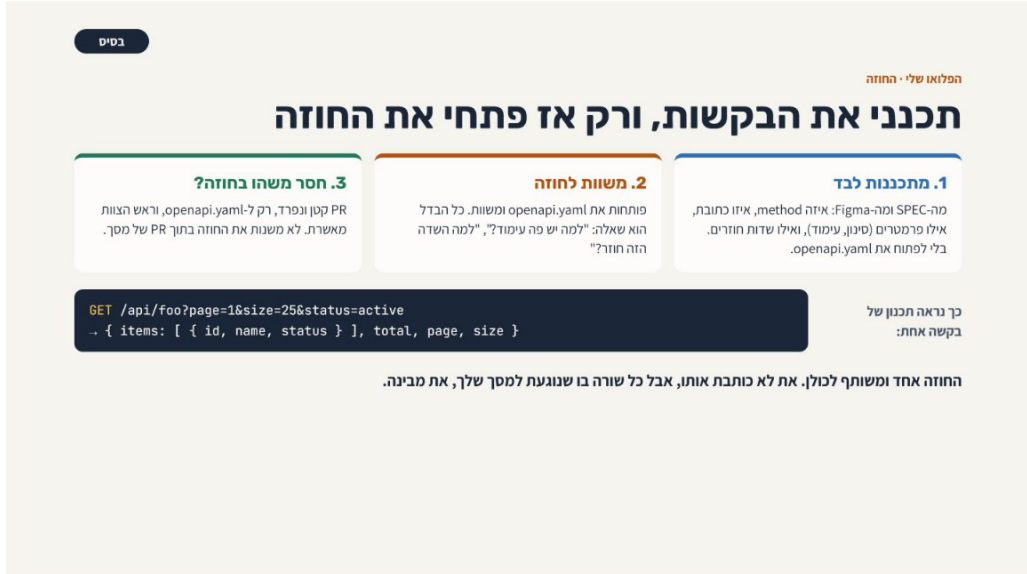
זה המסר המרכזי של החלק, ואולי של כל השיעור. להגיד אותו בטון רגוע, לא מאשים: "השתמשתן ב-AI, וזה בסדר. זה כלי של מפתחות. אבל עד עכשיו הוא עשה בשבילכן גם את החלק של ההבנה, - כל שלב נבנה על הקודם. בסרבר א' צריך לדעת מה החוזה של המסך, אילו אוספים הוא קורא, ומי רשאית. מי שלא מבינה את למה זה חשוב דווקא עכשיו: ואת זה הוא לא יכול לעשות במקומכן." - כשמשהו נשבר ב-dev אחרי מיזוג, רק מי שמבינה את - בריוויו, ראש הצוות יכולה לשאול "תסבירי לי את השורה הזו". מהיום, תשובה כמו "ה-AI כתב את זה" לא מספיקה. המסך שלה תיתקע. שלושת הכרטיסים הם ההגדרה של "להבין". בשקפים הבאים: איפה מוצאים את המידע (31), כרטיס המסך שלי (32), לתכנן את הבקשות לפני החוזה (33), ואיך עובדים הקוד שלה יכולה לתקן אותו. אם הזמן קצר: השקף הזה, כרטיס המסך (32), החוזה (33) וה-AI (34) הם החובה. את שקף המקורות (31) אפשר לתת כשיעורי בית. נכון עם AI (34).



1. Jira: המשימה של השלב הנוכחי, עם איפה כל אחד נמצא: הרבה בנות פתחו רק את המשימה ב-Jira והדביקו אותה ל-AI. השקף הזה מראה שיש שבעה מקורות, וכל אחד עונה על שאלה אחרת.
2. ריפו school-docs, הקובץ index.html, ומשם "אפיון" ואז העמוד של המסך (spec/screens/SCR-X01.html). יש בו שרטוט סכמטי וסעיף "שרת ונתונים", שהוא בדיוק מה קישורים לשאר.
3. Figma: הפריים של המסך, כולל מצבי ריק. school-client/src/screens/X01/SPEC.md. הסעיפים "API" ו"אוספים (Mongoose)" אומרים מה המסך קורא ומה בבעלותו. שצריך לסרבר א'.
4. school-docs/flows/process-flows.html: התהליכים (למשל, איך משהו עובר ממסך למסך ומתפקיד 5. school-docs/flows/screens-flow.html: כל המסכים והחיבורים ביניהם. ושגיאה.
להדגים חי על מסך אחד (כדאי של אחת הבנות): לפתוח את ארבעת הראשונים זה לצד זה ולהראות שהם מספרים את אותו סיפור מזוויות 7. בקוד: מטריצת ההרשאות, החוזה, והמודלים. לתפקיד).
שונות. כשיש סתירה, היקף v1-lite קובע.



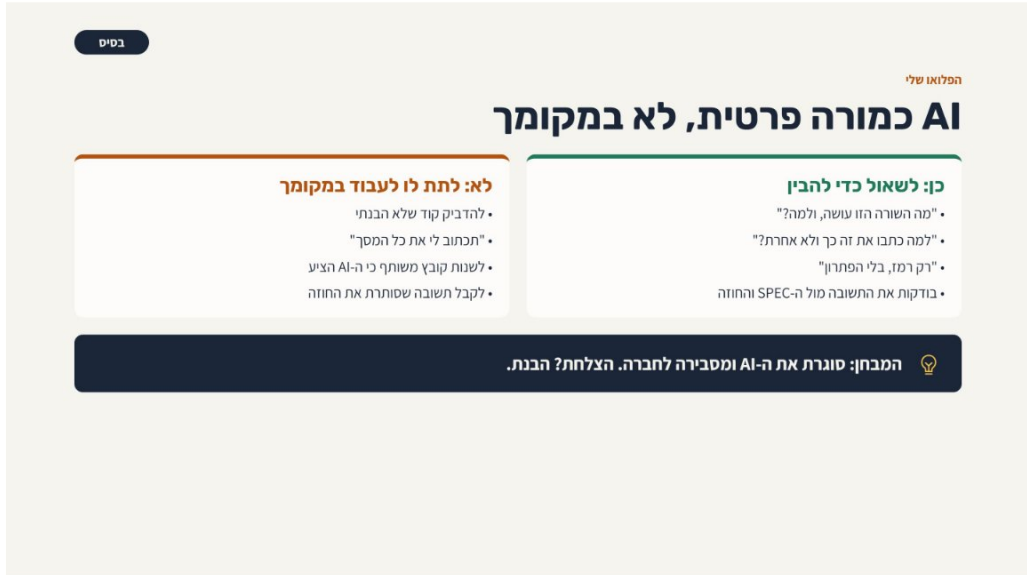
צבעי המספרים: כחול (1–4) = הצד של המשתמשת והקליינט. כתום (5–7) = הצד של הסרבר והמסד. ירוק (8) זה הכלי המעשי של החלק. כל בת ממלאת כרטיס כזה לכל אחד משני המסכים שלה.
- המשימה: למלא את שני - להראות דוגמה מלאה על מסך אחד ביחד (למשל, מסך ההדרכה של ההודעות, או מסך של אחת הבנות שמסכימה). איך להעביר: = איפה אני נמצאת בתהליך.
- הכלל: במילים שלך. מותר לחפש ולשאול, אבל התשובה כתובה בניסוח שלך. הכרטיסים לפני שמתחילים לכתוב קוד לסרבר א'. מוסיפים אותם כתגובה במשימה ב-Jira, כך שראש הצוות רואה.
מה זה נותן לך כמורה: כרטיס עם תשובות חסרות או מבולבלות מראה בדיוק איפה הבת לא מבינה. למשל, מי שלא יודעת לענות על 6 אם העתקת ניסוח מה-SPEC בלי להבין, זה לא נחשב.
שאלה 8 חשובה במיוחד: היא מחברת בין מה שכבר נעשה (קליינט, סכמות) לבין מה שבא. רבות לא (אוספים) תתקשה בסרבר א'. אפשר לעבור על הכרטיסים לפני שמחלקים את משימות סרבר א'.
יודעות שהסכמות שהן כתבו הן בדיוק מה שה-endpoint שלהן יקרא.



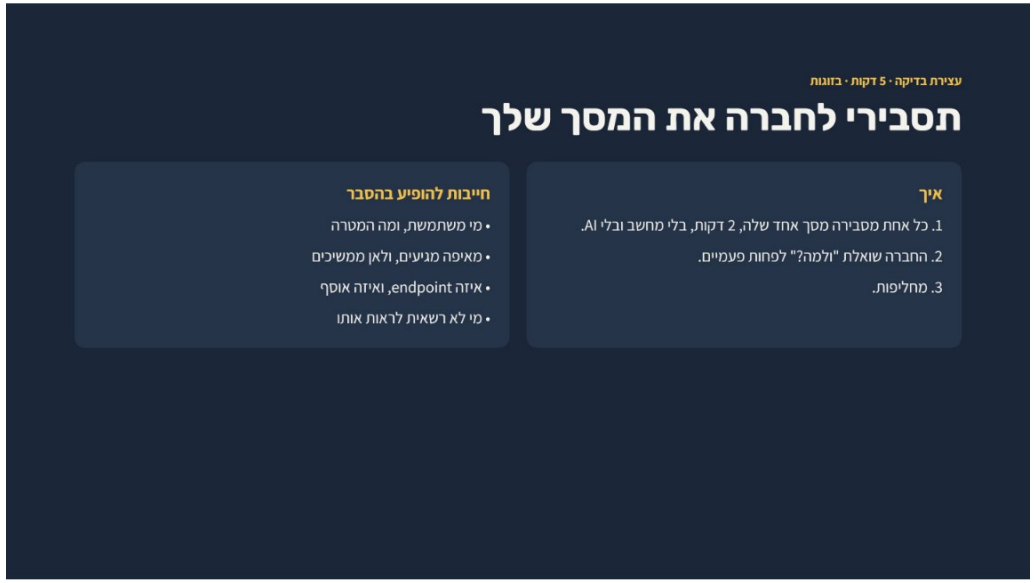
למה החוזה כתוב מראש, ולא כל אחת כותבת את שלה: זו שיטה מקובלת בתעשייה שנקראת API design-first. החוזה מתוכנן ומאושר לפני שכותבים קוד, בדרך כלל על ידי אדם אחד או צוות קטן. אבל לקבל חוזה מוכן בלי לחשוב זה בדיוק "לעבוד אם 13 בנות עורכות את אותו קובץ במקביל, יוצאים שמות שונים לאותו דבר, צורות תשובה שונות, והרבה קונפליקטים. לכן החוזה נשאר אחד.

- להדגים על מסך ההודעות: לשאול את הכיתה "מה המסך הזה צריך מהשרת?", לכתוב על הלוח, ורק אז לפתוח את החוזה איך מעבירים: כמו תוכי". לכן התרגיל: קודם לחשוב לבד, ואז להשוות.
- כשיעורי בית לפני סרבר א': כל בת כותבת את הבקשות של כל אחד משני המסכים שלה, בפורמט של הדוגמה, ומוסיפה אותן לכרטיס המסך ב-Jira (שאלה 5 בכרטיס). אחר כך היא ולהשוות.
- מה זה מגלה: בת ששכחה עימוד או הרשאות לא חשבה על הצד של הסרבר. בת שמתכננת שדות שלא קיימים במודל לא חיברה בין הסכמות למסך. מוסיפה שורה: "מה היה שונה בחוזה, ולמה".

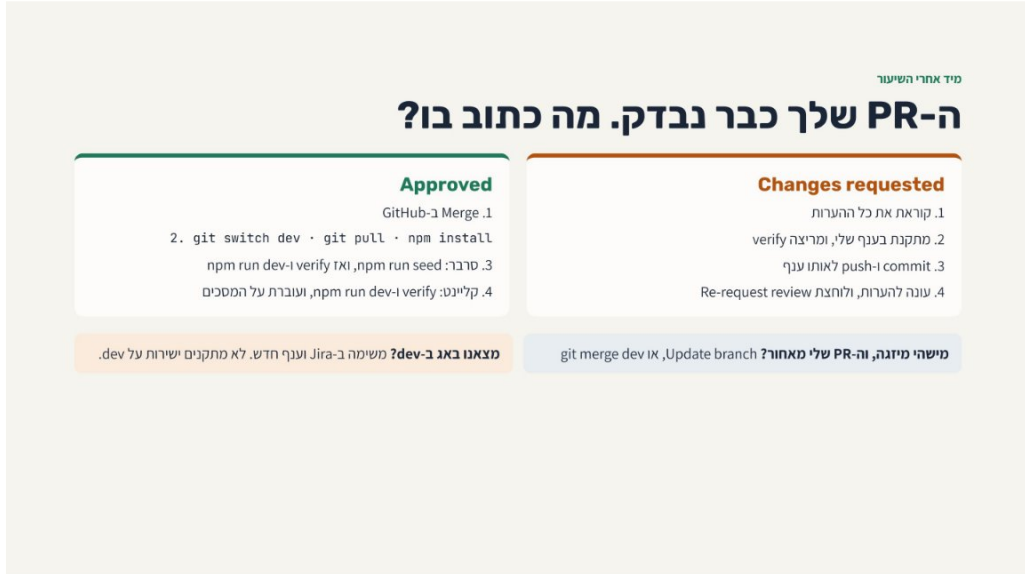
שלב 3: אם בת מוצאת שבאמת חסר משהו בחוזה, זה מצוין. היא למדה לחשוב כמו מתכננת API. היא פותחת PR נפרד ל-openapi.yaml בלבד, וראש הצוות מחליטה. אחרי מיזוג, בקליינט כלי עזר: ב-VS Code, Ctrl+Shift+F על הכתובת (למשל /api/foo) מוצא אותה גם בחוזה, גם ב-fooApi.ts בקליינט, וגם ב-routes.ts בסרבר. כך רואים את כל מריצים npm run contract:sync. המסלול של הבקשה בקבצים של הבת.



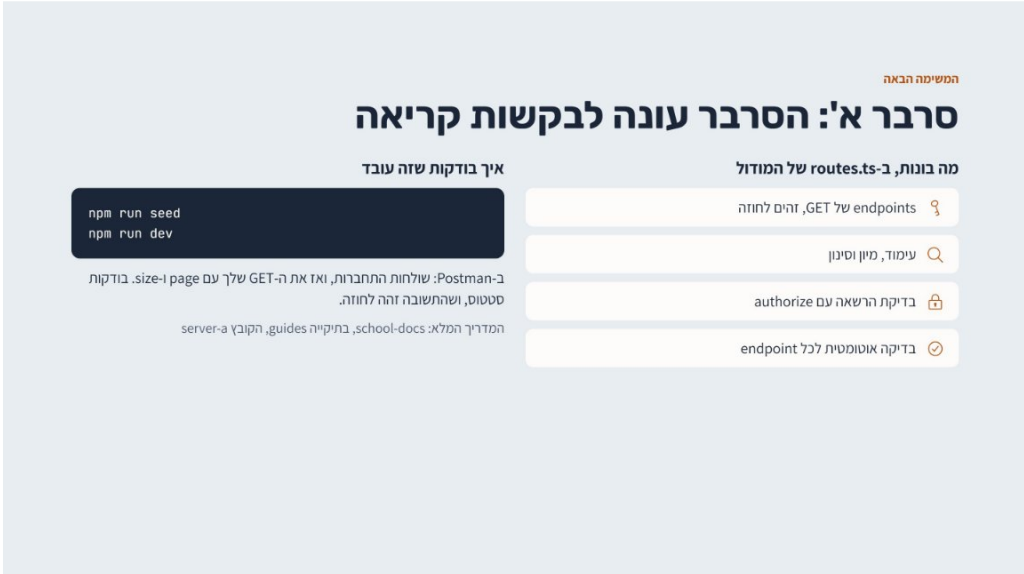
- להדביק קטע קוד צד ימין, דוגמאות לבקשות טובות: המטרה היא לא לאסור AI. מפתחות מקצועיות משתמשות בו כל יום. המטרה היא לשנות את התפקיד שלו: מ"כותב במקומי" ל"מסביר לי".
- לבדוק את התשובה: ה-AI לא מכיר את - לבקש רמז קטן, ולנסות לבד לפני שמבקשים עוד. - לבקש לבדוק הבנה: "הסברתי לעצמי שהקוד הזה עושה X. נכון?" ולבקש הסבר שורה אחרי שורה.
- קוד צד שמאל, למה זה מסוכן: הפרויקט שלנו. הוא לא יודע מה בחוזה, מה ב-SPEC ומה הכללים (למשל, רק רכיבים מ-components/ui). הרבה פעמים הוא ממציא שדה או כתובת שלא קיימים.
- AI מציע לפעמים "לתקן" קובץ משותף כדי שהקוד - "כל המסך" מה-AI כמעט תמיד לא תואם לחוזה, ל-Figma או לכללי ה-lint. שהודבק בלי הבנה הוא קוד שאי אפשר לתקן או להסביר בריוויו.
המבחן בתחתית הוא גם מה שעושים עכשיו, בעצירת הבדיקה. שלך יעבוד. זה שובר לאחרות.



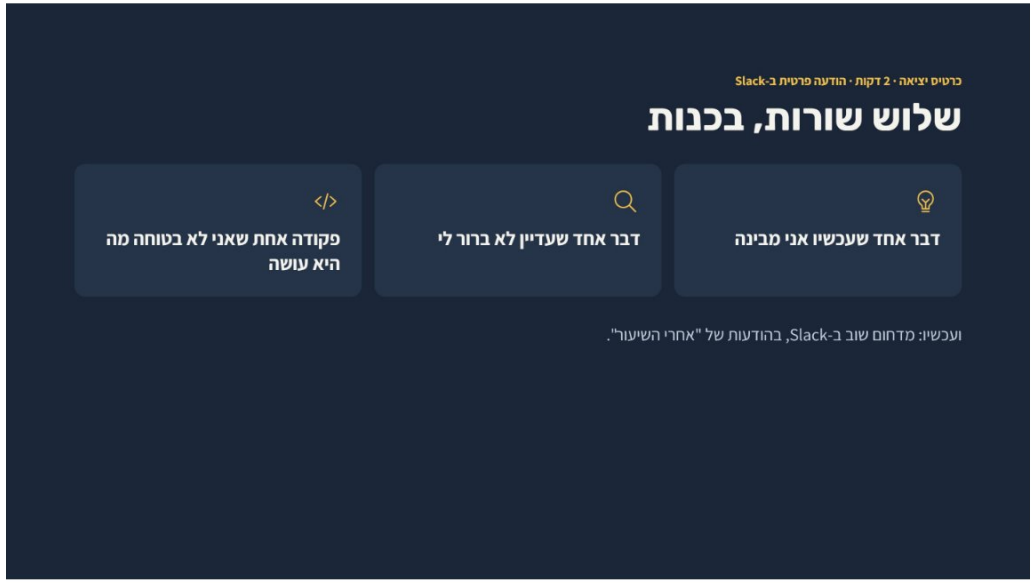
- הסבר שהוא ציטוט של בזמן שהן מדברות, לעבור בין הזוגות ולהקשיב. לא לתקן, רק לשמוע. סימנים לחוסר הבנה: זו עצירת הבדיקה של החלק, והיא גם המבחן מהשקף הקודם: מסבירות בלי AI.
- - לא יודעת איזה אוסף המסך קורא (לא חיברה בין משימת הסכמות למסך). - לא יודעת מאיזה מסך מגיעים למסך שלה (לא פתחה את תרשים הזרימה). ה-SPEC, בלי יכולת לענות על "ולמה?".
בנות שהתקשו: כרטיס המסך אחרי 5 דקות: לשאול שתיים-שלוש בנות בכיתה שאלה אחת מהרשימה, על המסך של עצמן. מתבלבלת בין מה שהקליינט עושה למה שהסרבר עושה (לחזור לשקף 4).
(שקף 32) הוא שיעור הבית שלהן, וכדאי לעבור עליו איתן אישית לפני שמחלקים את משימות סרבר א'.



ראש הצוות כבר סקרה את כל ה-PR-ים. כל בת פותחת את שני ה-PR שלה (קליינט וסרבר) ורואה את זה החלק המעשי, אחרי השיעור. כדאי להשאיר את השקף הזה פתוח על המקרן בזמן העבודה. עמודת Changes requested: לוודא שהן לא פותחות PR חדש, ושהן לוחצות Re-request review בסוף. הסטטוס, ובכל אחד הולכת לעמודה המתאימה. ייתכן שאחד מאושר ובשני יש הערות. - seed: רק npm run seed. - Merge: הכפתור הירוק בתחתית ה-PR. אחרי המיזוג GitHub מציע Delete branch, וזה בסדר למחוק. עמודת Approved: אחרת ראש הצוות לא יודעת שסיימו. - קליינט: כל עוד הקליינט עובד השירות MongoDB כבר רץ במחשב. לא צריך Docker ולא שום פקודה לפני. להריץ פעמיים ולוודא שאין כפילויות. אם יש בעיה, לחזור לשקף 18 (המסד המקומי). "מישהי מיזגה, וה-PR שלי מאחור": כשמישהי ממזגת ל-dev, PR-ים אחרים עלולים להראות "out-of-date" או עם Mock, בודקות שהמסכים של כולן מוצגים ושאין שגיאות בקונסול (F12). קונפליקט. בפעם הראשונה כדאי לפתור את זה ביחד על המקרן. Update branch הוא כפתור ב-GitHub שעושה את ה-merge בשבילך, כשאין קונפליקט. אחרי עדכון כזה, להריץ git pull בענף לשים לב לבנות שבשלב 2 לא מריצות git status קודם. זה המקום לחזק את ההרגל. שלך במחשב.



- לחבר למסלול של בקשה (שקף 6): בסרבר א' אתן בונות את שלבים 3 עד 6 בשביל המשאב שלכן. במשימת הקליינט ה-Mock "העמיד פנים" שהוא עושה את זה. עכשיו בונות את הדבר האמיתי. "זהים לחוזה": אותה כתובת, אותם פרמטרים, ואותה צורת תשובה כמו ב-openapi.yaml. אם הקליינט ציפה ל-items, total, page ו-size, זה מה שהסרבר מחזיר. יש כבר כלי עזר ב-src/utils/pagination.ts. הרשאה: authorize('X01', ...) כבר מופיע בקובץ שנוצר. לא מוחקים - עימוד: לא מחזירים את כל האוסף. page ו-size מגיעים ב-query, ומשתמשים ב-skip וב-limit. Postman: כותבות את הבקשה לפי החוזה, עם טוקן מבקשת ההתחברות. ההוראות במדריך - בדיקה: לכתוב בדיקה לכל endpoint (ראו routes.db.test.ts במודול announcements). אותו שאלה לסיום החלק: "מה יקרה אם תשכחי את authorize?" תשובה: כל משתמשת מחוברת תוכל לקרוא את הנתונים, גם בלי הרשאה. זו פרצת אבטחה, ולכן זה נבדק ברירוויו. server-a.



- לקבץ את ה"לא ברור לי" לפי נושא. נושא מה עושים עם התשובות (אחרי השיעור): למה הודעה פרטית: בנות שמתביישות לשאול בכיתה יכתבו בכנות. את מקבלת תמונה אמיתית של כל אחת.
- "פקודה שאני לא בטוחה": אם רובן כותבות את אותה פקודה (בדרך כלל merge, pull או push -u), זה המקום לתרגול קצר שחוזר אצל שלוש בנות ומעלה, פותחים בו את המפגש הבא (5 דקות).

מדחום שוב: לחזור על אותם שישה נושאים מהשקף השלישי. אם המספרים - בת שכתבה "הכול ברור", אבל במשימה היה לה קשה: לבדוק איתה אישית בזמן המיזוג. בטרמינל בזמן המיזוג היום. עלו, להגיד את זה בקול. זה נותן להן ביטחון לפני סרבר א'.